

Boundary Audit Summary

Prepared by Cyfrin

Version 2.2

Lead Auditors

Immeas

BengalCatBalu

January 10, 2026

Contents

1 About Cyfrin **2**

2 About This Document **2**

3 Disclaimer **2**

4 Copyright and License **2**

5 Audit Overview **3**

6 Audit Scope **3**

7 Risk Classification **4**

8 Protocol Summary **4**

8.1 Key Components 4

8.2 Core Flows 4

9 Attestation **5**

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Boundary Finance on Boundary. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.2 dated January 10, 2026, was delivered to Boundary Finance, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Boundary Finance or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 9 days, the Cyfrin team conducted an audit on the Boundary code provided by Boundary Finance. The findings consist of 3 Low severity issues with the remainder being informational.

Summary

Project Name	Boundary
Repository	boundary-protocol-ethereum
Commit	99d752a795c0...
Fix Commit	b6fe3116aa1d...
Audit Timeline	Dec 29th - Jan 8th, 2026
Methods	Manual Review, Invariant testing

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	3	2	1
Informational	9	6	3
Gas Optimizations	0	0	0
Total	12	8	4

6 Audit Scope

The audit scope was limited to:

```
src/access/AccessControlDefaultAdminRulesEnumerable.sol
src/minting/USBDMinting.sol
src/rewards/StakingRewardsDistributor.sol
src/staking/sUSBD.sol
src/staking/USBDSilo.sol
src/token/USBD.sol

// deployment-related
script/initial/Deploy.s.sol
src/deployers/DeploymentOrchestrator.sol
src/deployers/sUSBDDeployer.sol
src/deployers/USBDDeployer.sol
```

Commits 18eff23, a88aee4, f8dd7f6 and e955a3b, made during the audit, were also reviewed.

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

Boundary is releasing a new USD stablecoin, USBD, together with staking capabilities. USBD is minted and redeemed against whitelisted ERC-20 collateral (intended to be stablecoins) via an off-chain engine that submits EIP-712 orders to USBDMinting. On mint, collateral is transferred directly from the benefactor to approved custodians via a configurable route, and USBD is minted to the beneficiary. On redeem, USBD is burned from the benefactor and collateral is transferred out of the minting contract's redemption buffer to the beneficiary.

Staking is provided via sUSBD, an ERC-4626 vault over USBD with linear vesting of rewards transferred into the vault. It can also operate in an optional cooldown-based withdrawal mode using a USBDSilo.

Rewards are distributed through StakingRewardsDistributor, which holds collateral and uses it to mint USBD through USBDMinting; the newly minted USBD is then transferred into sUSBD as rewards.

8.1 Key Components

- USBD (ERC-20): the stablecoin token.
- USBDMinting: EIP-712 order execution, allowlisted assets, custodians and benefactors. Supports EOA signatures and ERC-1271 for contract wallets.
- sUSBD (ERC-4626 vault): USBD staking shares; supports:
  - Standard mode: direct withdraw/redeem when cooldown is disabled
  - Cooldown mode: burn shares into USBDSilo, then claim after the timer
  - Linear vesting: rewards increase share value gradually
- USBDSilo: holds USBD during cooldown.
- StakingRewardsDistributor: holds collateral, mints USBD with it through USBDMinting, then transfers the minted USBD into sUSBD as rewards.

8.2 Core Flows

- Mint (collateral to USBD): a user obtains a quote off-chain and signs an EIP-712 mint order; the engine submits it; collateral is transferred from the benefactor directly to custodian destinations per route and USBD is minted to the beneficiary.
- Redeem (USBD to collateral): a user signs a redeem order and the engine submits it; USBD is burned from the benefactor and collateral is transferred from the minting contract's buffer to the beneficiary.
- Stake (USBD to sUSBD): users deposit USBD into sUSBD to receive shares at the prevailing exchange rate.
- Unstake: in standard mode users withdraw or redeem directly; in cooldown mode shares are burned and the corresponding USBD moves to USBDSilo, to be claimed after the cooldown ends.
- Rewards distribution (custody yield to sUSBD): stable collateral is sent to StakingRewardsDistributor, used to mint USBD through USBDMinting, and the USBD is pushed into sUSBD, starting a new linear vesting period.

9 Attestation

Cyfrin attests that it performed a security review of the Boundary code provided by Boundary Finance.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-01-10-cyfrin-boundary-v2.2.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Boundary Finance has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=84b01a866fa4bc3943aadd594cb935931865fa9e86d98c5f3ed988b3de2ace18
F=2026-01-10-cyfrin-boundary-v2.2-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.